Practical - 3

Practical Title: Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

Problem Definition: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

A. Theory

1. How does the layout adapt for mobile, tablet, and desktop screens?

Answer :
The layout adapts fluidly. Instead of strict media queries, it uses responsive CSS properties like flex-wrap: wrap and min-width: 180px (seen in .feature-box). This allows elements to automatically stack on mobile/tablet screens and sit side-by-side on desktop screens.

2. Which layout technique is used and why?

Answer :
CSS Flexbox (display: flex;) is the primary layout technique used (specifically for .features and form containers). It is used because it provides a modern, efficient way to align elements, distribute space, and handle dynamic wrapping without relying on complex and outdated float rules.

3. Are typography, spacing, colors, and contrast readable?

Answer :
Yes. Typography uses clean, modern fonts (Segoe UI, Arial). Spacing is consistent (e.g., padding: 24px, gap: 15px). The color palette uses a warm, earthy theme (browns on beige), providing excellent contrast and high readability between the dark text (#3a2a1a) and light backgrounds (#fffdfa).

4. Is the framework used consistently without unnecessary inline CSS?

Answer :
Yes, perfectly**.** Pure CSS3 is used consistently via a single external stylesheet (../CSS/script.css). A scan of all HTML files in Practical 3 confirms there is **zero** inline CSS (no style="..." attributes used anywhere), ensuring a clean separation of structure and design.

B. Implementation

This practical focused on UI design and project organization.

The flat file structure was reorganized into a professional architecture using CSS, JS, and PAGES directories.

A central script.css stylesheet was created and linked to all pages, utilizing advanced CSS techniques like linear gradients, box shadows, Flexbox layouts, and hover effects to transform the raw HTML into a polished, modern interface.

C. Evidence of Work (Screenshots)

Description: home page of student hub portal

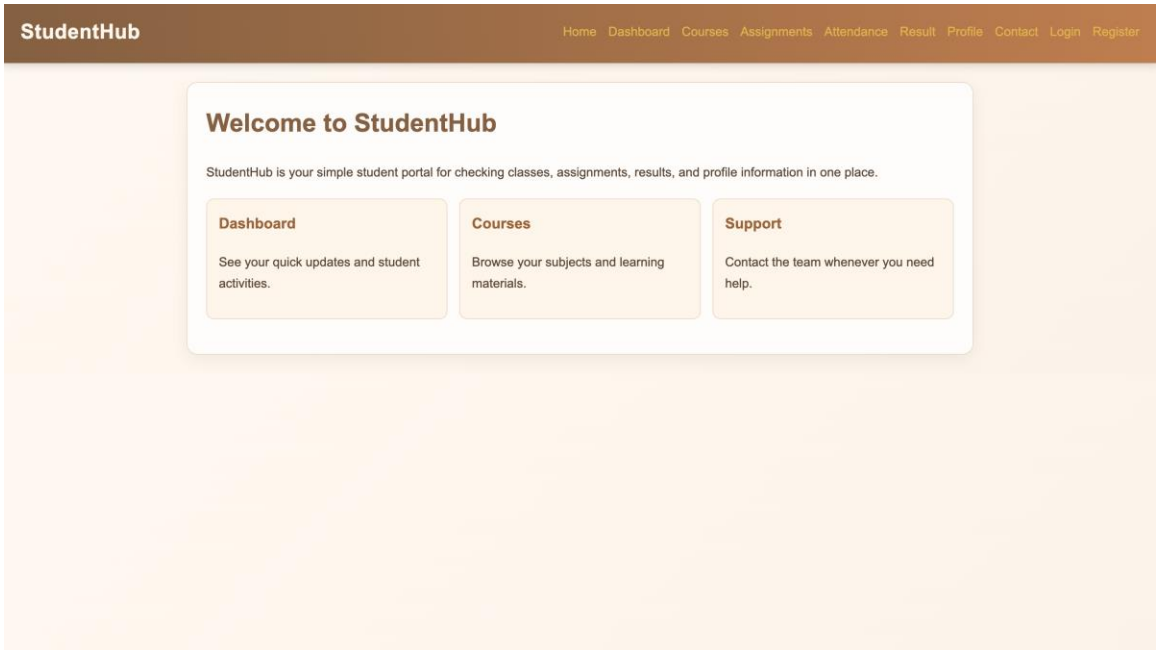


Figure 3.1

Description: login page

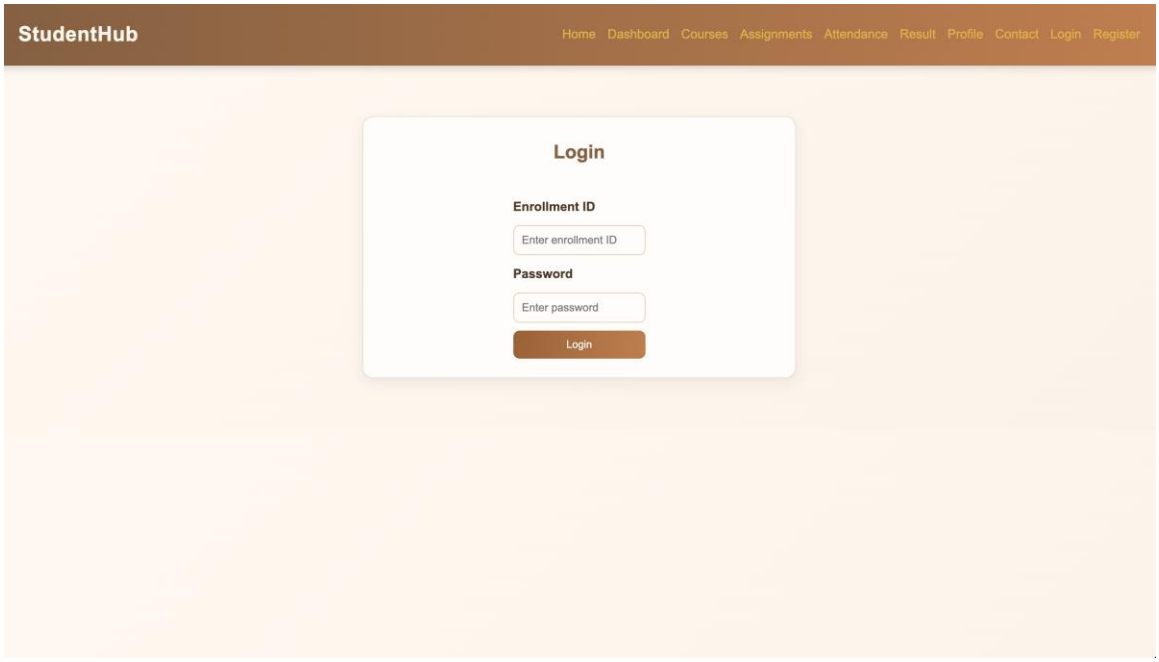


Figure 3.2

Description: register page

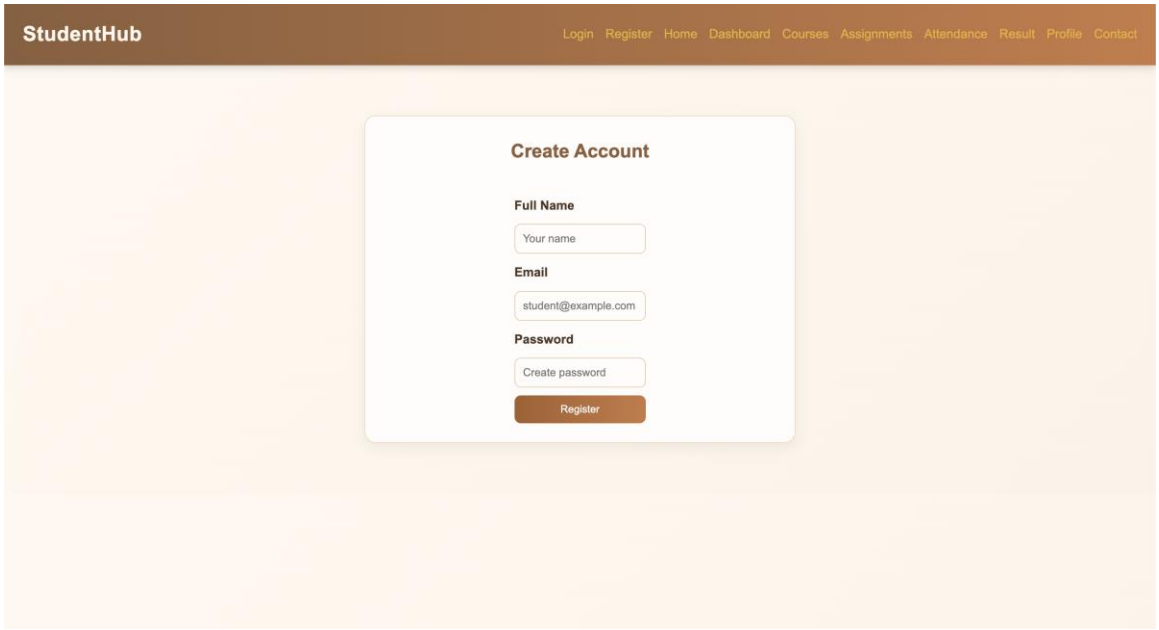


Figure 3.3

Description: profile page

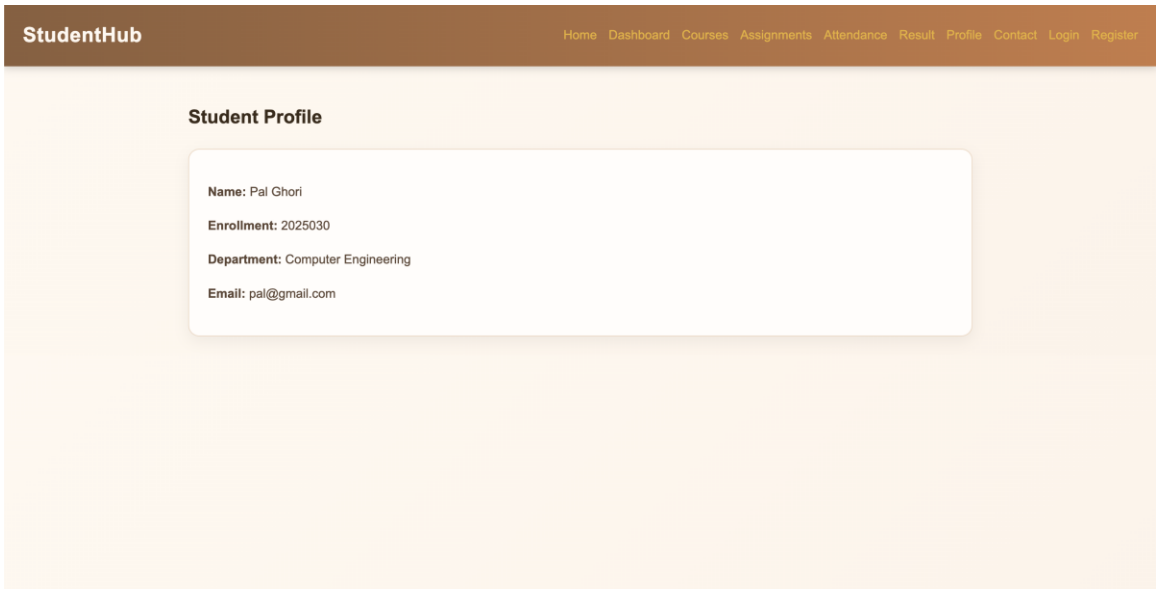


Figure 3.4

Description: contact page

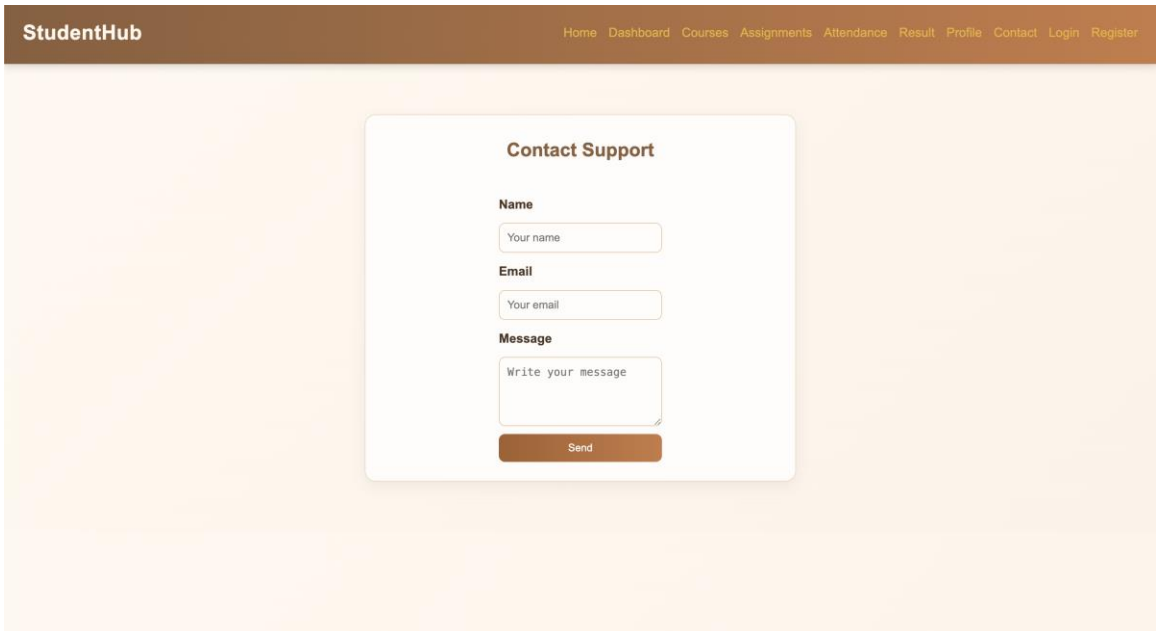


Figure 3.5

Description: result page

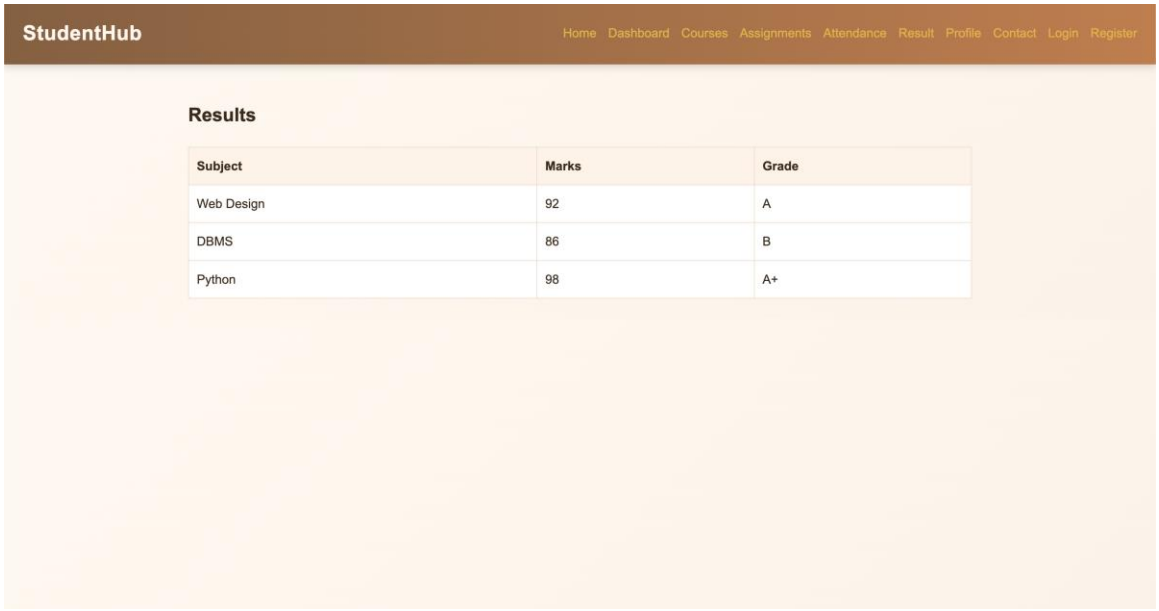


Figure 3.6

Description: attendance page

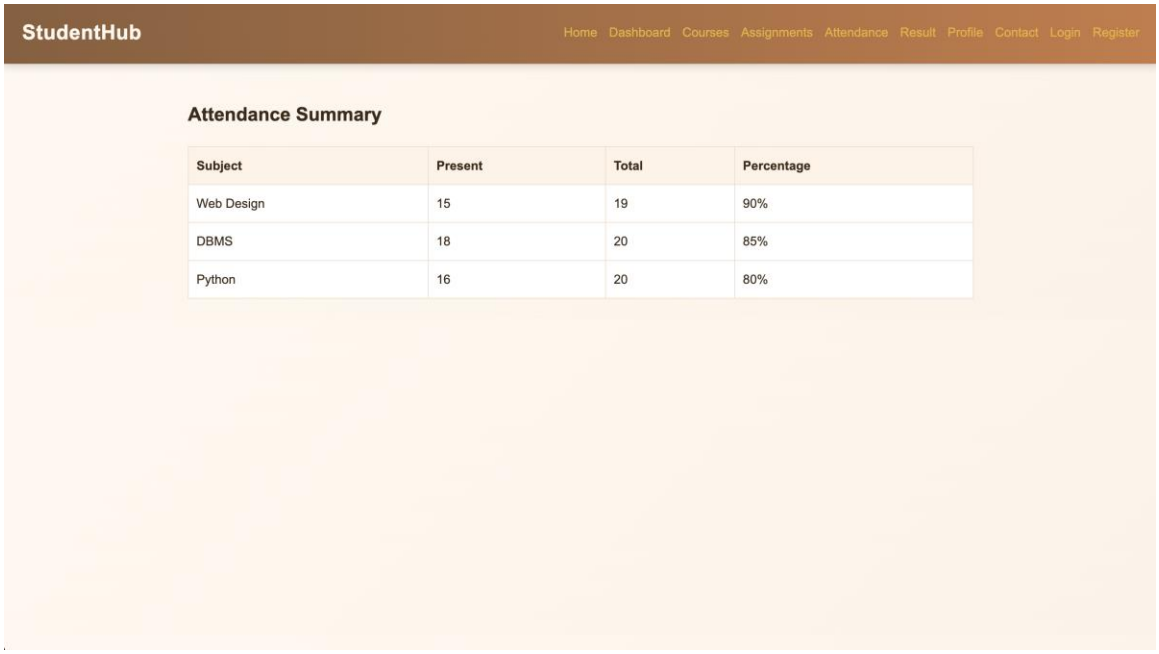


Figure 3.7

Description: assignment page

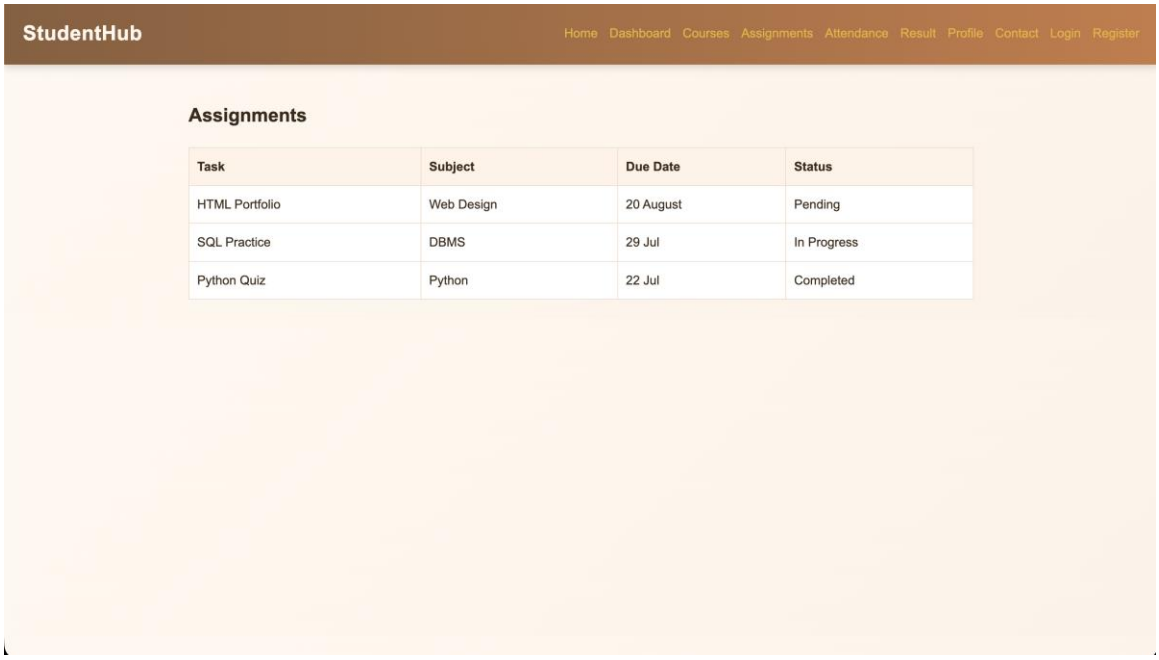


Figure 3.8

Description: courses page

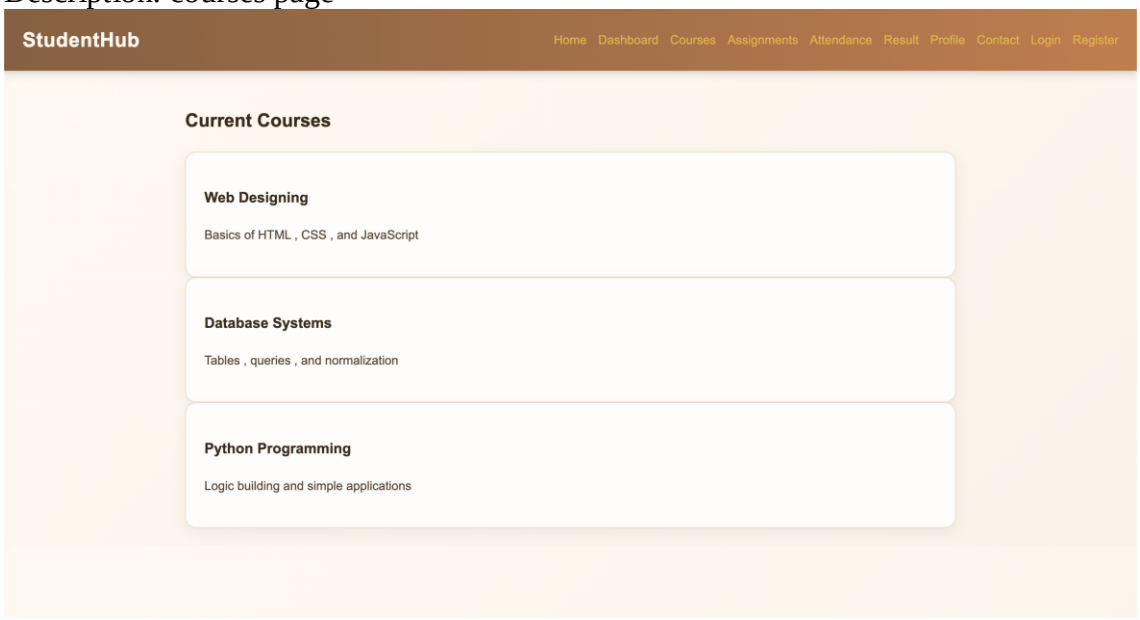


Figure 3.9

Description: dashboard page

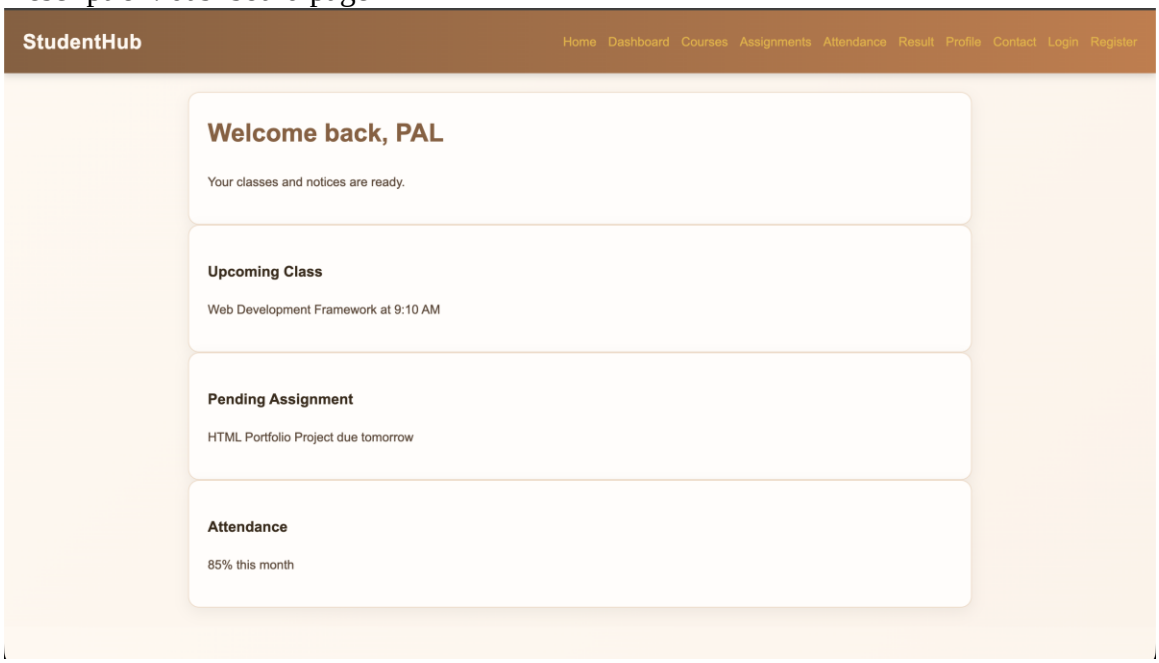


Figure 3.10

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%203

E. Challenges Faced:

In Practical 3, moving the raw HTML into a structured directory architecture (PAGES, CSS, JS) broke all existing file links, which was challenging to fix by updating every ../ relative path.

When writing script.css, applying Flexbox to the feature boxes and form containers was tricky; getting the items to align and wrap correctly required constant tweaking.

Additionally, implementing consistent styling for tables and navigation bars across all pages meant I had to rely heavily on CSS classes and global rules, which was a significant shift in thinking.

Finally, debugging conflicting CSS rules where styles accidentally overrode each other took considerable time.

F. Learning Outcomes:

Learned to structure a web project professionally using dedicated directories.

Gained experience applying aesthetic properties like gradients, shadows.

Learned how to resolve relative path issues when linking external stylesheets.

Understood the importance of a single global stylesheet for maintaining UI consistency.

Faculty Remarks: